

Basi di dati - Soluzioni ragionate degli appelli

Appelli 2024-2025, con pattern e trappole ricorrenti

Giugno 2026

Indice

14 gennaio 2025 - Registry	2
Esercizio 1 - Reverse engineering ER	2
Esercizio 2 - Query	3
1. Coppie padre-figlio	3
2. Documenti non scaduti per ogni persona, incluso zero	3
3. Persone con il maggior numero di certificati diversi	3
4. Algebra: persone senza documenti validi	3
5. Algebra: persone che hanno richiesto tutti i certificati	3
Esercizio 3 - Need-to-know e maximized sharing	3
Esercizio 4 - Proprietà della decomposizione	3
Esercizio 5 - Violazioni DDL osservabili	4
16 luglio 2024 - Flight	4
Esercizio 1 - Reverse engineering ER	4
Esercizio 2 - Query	4
1. Aeroporti con più partenze che arrivi	4
2. Velivoli con il maggior numero di voli	4
3. Coppie di passeggeri sullo stesso volo	5
4. Voli con più dell'80% dei posti occupati	5
5. Algebra: clienti mai partiti da Milano Linate	5
Esercizio 3 - Dirty read	5
Esercizio 4 - Grafo delle autorizzazioni	5
Esercizio 5 - Risposte brevi	6
10 giugno 2025 - eCommerce	6
Esercizio 1 - Soglia	6
A-D	6
E. Risultato del RIGHT JOIN	6
F. Algebra: ordini che contengono laptop	6
Esercizio 2 - Query	6
1. Clienti senza ordini in spedizione	6
2. Spesa complessiva, incluso zero	6
3. Categoria con più prodotti	7
4. Stessa quantità dello stesso prodotto in due ordini diversi	7
5. Algebra: cliente con ordine travelling più recente	7
Esercizio 3 - Da ER a relazionale	7
Esercizio 4 - Normalizzazione	7
1 luglio 2025 - Library	8
Esercizio 1 - Soglia	8
A-D	8
E. Divisione	8
F. Vero/falso	8
Esercizio 2 - Query	8
1. Clienti che non hanno mai preso libri romantici	8
2. Prestito iniziato nella data di una restituzione	8

3. Più prestiti della media nel 2025	9
4. Libri di Pulixi senza posizione	9
5. Algebra: almeno un Carlotto e almeno un Lansdale	9
Esercizio 3 - Gerarchia (P,E)	9
Accorpamento nel padre	9
Mantenimento di padre e figli	9
Sole entità figlie	10
Esercizio 4 - Grafo e revoca	10
22 luglio 2025 - Festival	10
Esercizio 1 - Soglia	10
A-D	10
E. Risultato del LEFT JOIN	10
F. Effetto del DELETE	11
Esercizio 2 - Query	11
1. Film con il maggior numero di premi	11
2. Coppie di artisti, stesso premio, anni consecutivi, film diversi	11
3. Artisti con premi in ogni categoria presente	11
4. Film proiettati in più di tre venue diverse	11
5. Algebra ottimizzata: film senza premiere	11
Esercizio 3 - Reverse engineering ER	12
Esercizio 4 - Cavallo di Troia	12

Errori che farebbero perdere punti in più appelli **12**

Le soluzioni privilegiano correttezza, gestione degli ex aequo e inclusione dei casi a zero. Dove una fotografia non rende leggibili tutti i dati, viene data la parte determinabile senza inventare valori.

Convenzioni:

- in algebra $p_x(R)$ rinomina la relazione R in x;
- TODAY indica la data corrente;
- "order" è quotato perché ORDER è parola riservata in SQL;
- negli appelli Library i riferimenti inglesi book/client sono refusi: nelle soluzioni si usano le tabelle dichiarate libro/cliente.

14 gennaio 2025 - Registry

Esercizio 1 - Reverse engineering ER

Schema concettuale:

- entità PERSONA: identificatore id, attributi name, birth_date opzionale;
- due associazioni ricorsive distinte su PERSONA, con ruoli espliciti: ha_padre(figlio,padre) e ha_madre(figlio,madre). Ogni figlio partecipa 0..1 a ciascuna; una persona può essere padre/madre di 0..N persone;
- entità DOCUMENTO: identificatore serial_n, attributi doc_type, released_by opzionale, release_date, expiry_date;
- associazione 1:N possiede: ogni documento appartiene a 1..1 persona, ogni persona possiede 0..N documenti;
- entità CERTIFICATO: identificatore id, attributo description;
- associazione N:M storica richiede tra persona e certificato, con attributo r_date. La tripla (r_date, persona, certificato) identifica un'occorrenza, quindi la stessa coppia può ricomparire in date diverse.

Il CHECK sui tipi di documento è un vincolo di dominio da annotare. Il DDL non permette di ricavare sesso, età o altri vincoli sui ruoli padre/madre.

Esercizio 2 - Query

1. Coppie padre-figlio

```
SELECT padre.name AS padre, figlio.name AS figlio
FROM person AS figlio
JOIN person AS padre ON padre.id = figlio.father;
```

2. Documenti non scaduti per ogni persona, incluso zero

```
SELECT p.id, p.name, COUNT(d.serial_n) AS documenti_validi
FROM person AS p
LEFT JOIN document AS d
  ON d.doc_owner = p.id
  AND d.expiry_date >= CURRENT_DATE
GROUP BY p.id, p.name;
```

Il filtro è nell'ON; con WHERE scomparirebbero le persone con zero.

3. Persone con il maggior numero di certificati diversi

```
WITH conteggi AS (
  SELECT p.id, p.name,
         COUNT(DISTINCT cr.certificate) AS n
  FROM person AS p
  LEFT JOIN certificate_request AS cr ON cr.person = p.id
  GROUP BY p.id, p.name
)
SELECT id, name, n
FROM conteggi
WHERE n = (SELECT MAX(n) FROM conteggi);
```

Questa versione include anche il caso limite in cui nessuno ha richieste: tutte le persone sono a pari merito con zero.

4. Algebra: persone senza documenti validi

```
Validi =  $\sigma_{\{expiry\_date \geq TODAY\}}(document)$ 
ConValidi =  $\pi_{\{id,name\}}(person \bowtie_{\{person.id = Validi.doc\_owner\}} Validi)$ 
Risultato =  $\pi_{\{id,name\}}(person) - ConValidi$ 
```

5. Algebra: persone che hanno richiesto tutti i certificati

```
Richieste =  $\pi_{\{person,certificate\}}(certificate\_request)$ 
TuttiCert =  $\rho_{\{certificate \leftarrow id\}}(\pi_{id}(certificate))$ 
IdPersone =  $Richieste \div TuttiCert$ 
Risultato =  $\pi_{\{id,name\}}(person \bowtie_{\{id=person\}} IdPersone)$ 
```

Esercizio 3 - Need-to-know e maximized sharing

Need-to-know concede solo gli accessi indispensabili: è restrittiva, sicura e tipica dei sistemi chiusi, ma può impedire accessi innocui. Maximized sharing concede il massimo accesso compatibile con le informazioni riservate: favorisce collaborazione e disponibilità, ma richiede più fiducia e offre meno protezione.

Esercizio 4 - Proprietà della decomposizione

- **Senza perdita:** il join delle proiezioni sulle relazioni decomposte ricostruisce esattamente ogni istanza lecita della relazione originale, senza tuple spurie.
- **Conservazione delle dipendenze:** tutte le DF originarie sono inferibili dall'unione delle DF verificabili nelle singole relazioni decomposte; non serve un join per controllarle.

Per una decomposizione binaria $R1, R2$, il join è senza perdita se $R1 \bowtie R2 \rightarrow R1$ oppure $R1 \bowtie R2 \rightarrow R2$ rispetto alla chiusura delle DF.

Esercizio 5 - Violazioni DDL osservabili

- R1: doc_type = 'carta id.' non appartiene all'insieme ammesso. La data di scadenza precedente al rilascio **non** viola il DDL: manca quel CHECK.
- R2: doc_owner = '5B' non appartiene al dominio integer.
- R3: il tipo vuoto non è fra i valori ammessi; se la cella rappresenta NULL, viola anche NOT NULL. Se rappresenta '', viola il solo CHECK.
- R4: doc_type = 'carta id.' non è ammesso. released_by può essere nullo.
- R5: doc_type = 'CF' non è ammesso ('cod. fiscale' è il valore previsto).

Non è possibile verificare la FK doc_owner → person.id senza vedere l'istanza di person. Non ci sono duplicati o null osservabili in serial_n.

16 luglio 2024 - Flight

Esercizio 1 - Reverse engineering ER

- CITY è entità con identificatore name e attributo country opzionale.
- AIRPORT è entità identificata da (name, city), collegata a CITY: ogni aeroporto è in esattamente una città; una città ha 0..N aeroporti. Si può rappresentare AIRPORT come entità debole con identificatore parziale name rispetto a CITY.
- AIRCRAFT è entità con identificatore id e attributi model, seats, attributo booleano non leggibile, brand con dominio Boeing/Airbus.
- FLIGHT è entità con identificatore code. Ha due associazioni distinte e totali verso AIRPORT, con ruoli partenza e arrivo; ogni aeroporto compare in 0..N voli per ciascun ruolo. L'associazione con AIRCRAFT è opzionale per il volo (0..1) e 0..N per il velivolo.
- CLIENT è entità con identificatore id.
- PASSENGER si minimizza in un'associazione N:M fra CLIENT e FLIGHT, con attributi price e seat; la PK impone una sola partecipazione della stessa coppia cliente-volo.

Il vincolo UNIQUE(air_company, aeroporto_partenza, datetime_partenza) è un identificatore alternativo di FLIGHT.

Esercizio 2 - Query

1. Aeroporti con più partenze che arrivi

```
SELECT a.name, a.city
FROM airport AS a
WHERE (
    SELECT COUNT(*)
    FROM flight AS f
    WHERE f.departure_airport_name = a.name
    AND f.departure_airport_city = a.city
) > (
    SELECT COUNT(*)
    FROM flight AS f
    WHERE f.arrival_airport_name = a.name
    AND f.arrival_airport_city = a.city
);
```

Le sottoquery correlate gestiscono correttamente anche gli zeri.

2. Velivoli con il maggior numero di voli

```
WITH conteggi AS (
    SELECT a.id, a.model, COUNT(f.code) AS n
    FROM aircraft AS a
    LEFT JOIN flight AS f ON f.aircraft = a.id
    GROUP BY a.id, a.model
)
```

```

)
SELECT *
FROM conteggi
WHERE n = (SELECT MAX(n) FROM conteggi);

```

3. Coppie di passeggeri sullo stesso volo

```

SELECT DISTINCT c1.id, c1.name, c2.id, c2.name
FROM passenger AS p1
JOIN passenger AS p2
  ON p1.flight = p2.flight
  AND p1.client < p2.client
JOIN client AS c1 ON c1.id = p1.client
JOIN client AS c2 ON c2.id = p2.client;

```

DISTINCT serve se la stessa coppia ha condiviso più voli.

4. Voli con più dell'80% dei posti occupati

```

SELECT f.code
FROM flight AS f
JOIN aircraft AS a ON a.id = f.aircraft
LEFT JOIN passenger AS p ON p.flight = f.code
GROUP BY f.code, a.seats
HAVING COUNT(p.client) > 0.8 * a.seats;

```

5. Algebra: clienti mai partiti da Milano Linate

```

DaLinate =  $\sigma_{\{departure\_airport\_name='Linate' \wedge$ 
            $departure\_airport\_city='Milano'\}}(flight)$ 
Partiti =  $\pi_{client}(passenger \bowtie_{\{passenger.flight=DaLinate.code\}} DaLinate)$ 
Risultato =  $\pi_{id}(client) - p_{\{id \in client\}}(Partiti)$ 

```

Esercizio 3 - Dirty read

T1 modifica X senza commit; T2 legge quel valore; T1 esegue rollback. T2 ha quindi usato un valore che non è mai diventato stabile. Si previene impedendo la lettura di scritture non confermate, per esempio con isolamento almeno READ COMMITTED e lock/versioning del DBMS.

Esercizio 4 - Grafo delle autorizzazioni

Grafo iniziale, per SELECT e INSERT separatamente:

```

Jannik --10,60--> Jannik
Jannik --20,60--> Carlos
Jannik --25,60--> Novak

```

```

SELECT: Novak --30,no60--> Carlos
INSERT: Novak --30,no60--> Alexander
SELECT/INSERT: Novak --30,60--> Hubert
SELECT/INSERT: Hubert --40,no60--> Alexander
SELECT/INSERT: Carlos --45,60--> Hubert

```

Dopo Jannik: REVOKE ALL ON R FROM Novak CASCADE:

- spariscono gli archi Jannik→Novak e tutti gli archi concessi da Novak;
- Carlos conserva entrambi i privilegi con GO grazie all'arco indipendente di Jannik al tempo 20;
- Hubert conserva entrambi con GO grazie a Carlos al tempo 45;
- le concessioni di Hubert ad Alexander al tempo 40 cadono: in quel momento Hubert dipendeva da Novak; la sorgente Carlos arriva solo al tempo 45;
- Alexander perde anche l'INSERT diretto da Novak.

Restano quindi la radice di Jannik, Jannik→Carlos (20,60) e Carlos→Hubert (45,60).

Esercizio 5 - Risposte brevi

1. Superchiave non minimale di flight: {code, air_company}.
 2. Con PK id e DF model → seats, brand, aircraft è in 2NF ma non in 3NF: model non è superchiave e seats/brand non sono primi. Quindi non è BCNF.
 3. No. La FK flight.aircraft non ha ON UPDATE CASCADE; un aggiornamento di un aircraft.id referenziato viene bloccato (NO ACTION). È integrità referenziale.
-

10 giugno 2025 - eCommerce

Esercizio 1 - Soglia

A-D

- A. Una chiave è una superchiave **minimale**; una superchiave può contenere attributi ridondanti.
- B. COMMIT.
- C. Nella classificazione delle dispense, le strutture ad accesso hash sono strutture primarie ad accesso calcolato.
- D. Politica need-to-know o minimo privilegio.

E. Risultato del RIGHT JOIN

Il RIGHT JOIN preserva tutti gli ordini. Si abbinano solo dettagli con quantity > 5; WHERE s.pid IS NULL conserva gli ordini senza tali dettagli.

- ordine 101: ha quantità 6 → escluso;
- ordine 102: quantità 4 e 1 → incluso;
- ordine 103: quantità 9 → escluso;
- ordine 104: quantità 6 → escluso.

Risultato: la sola tupla completa di order con oid=102, seguita dal valore NULL per s.oid (nei dati visibili: data terminante in 06-01 e stato delivered).

F. Algebra: ordini che contengono laptop

```
Laptop =  $\sigma_{\{name='laptop'\}}(product)$   
Risultato =  $\pi_{oid}(order\_detail \bowtie_{\{order\_detail.pid=Laptop.pid\}} Laptop)$ 
```

Usare l'esatto valore/maiuscole riportato nell'istanza se disponibili.

Esercizio 2 - Query

1. Clienti senza ordini in spedizione

```
SELECT c.*  
FROM client AS c  
WHERE NOT EXISTS (  
    SELECT 1  
    FROM "order" AS o  
    WHERE o.cid = c.cid  
    AND o.status = 'shipped'  
);
```

2. Spesa complessiva, incluso zero

```
SELECT c.cid, c.name,  
       COALESCE(SUM(od.quantity * p.price), 0) AS spesa  
FROM client AS c  
LEFT JOIN "order" AS o ON o.cid = c.cid  
LEFT JOIN order_detail AS od ON od.oid = o.oid
```

```

LEFT JOIN product AS p ON p.pid = od.pid
GROUP BY c.cid, c.name
ORDER BY spesa DESC;

```

3. Categoria con più prodotti

```

WITH conteggi AS (
    SELECT category, COUNT(*) AS n
    FROM product
    WHERE category IS NOT NULL
    GROUP BY category
)
SELECT category, n
FROM conteggi
WHERE n = (SELECT MAX(n) FROM conteggi);

```

4. Stessa quantità dello stesso prodotto in due ordini diversi

```

SELECT DISTINCT c.cid, c.name
FROM order_detail AS d1
JOIN order_detail AS d2
    ON d1.pid = d2.pid
    AND d1.quantity = d2.quantity
    AND d1.oid < d2.oid
JOIN "order" AS o1 ON o1.oid = d1.oid
JOIN "order" AS o2 ON o2.oid = d2.oid AND o2.cid = o1.cid
JOIN client AS c ON c.cid = o1.cid;

```

5. Algebra: cliente con ordine travelling più recente

$T = \sigma_{\{\text{status}='travelling'\}}(\text{order})$

$\text{MenoRecenti} = \pi_{\{o1.oid, o1.cid, o1.o_date, o1.status\}}(\sigma_{\{o1.o_date < o2.o_date\}}(\rho_{o1}(T) \times \rho_{o2}(T)))$

$\text{Recenti} = T - \text{MenoRecenti}$
 $\text{Risultato} = \pi_{\text{name}}(\text{client} \bowtie_{\{\text{client.cid}=\text{Recenti.cid}\}} \text{Recenti})$

Restituisce tutti gli ex aequo sulla data massima.

Esercizio 3 - Da ER a relazionale

La fotografia non consente di leggere con affidabilità tutti i nomi e le cardinalità. Il metodo valutato è:

1. una relazione per E1, E2, E3, E4, con le chiavi indicate dai pallini pieni;
2. per ogni associazione 1:N, migrare la PK del lato 1 nel lato N; aggiungere * se la partecipazione minima del lato N è 0;
3. per associazioni N:M (R1/R4, secondo le cardinalità leggibili), creare una relazione con PK composta dalle FK;
4. conservare l'attributo di R2 nella relazione dell'associazione oppure nel lato N se R2 è accorpabile;
5. minimizzare solo quando la cardinalità massima 1 rende possibile migrare la FK senza perdita.

Per esercitarsi sullo stesso tipo di domanda con dati completamente leggibili, usare gli esercizi ER degli appelli 01/07 e 22/07.

Esercizio 4 - Normalizzazione

$R(A, B, C, D, E, F, G, H)$
 $A \rightarrow B, C, D$
 $C \rightarrow D$
 $F \rightarrow G, H$
 $A, F \rightarrow E$

La chiave è AF: $(AF)^+ = ABCDEFGH$, mentre né A né F bastano. Una decomposizione che conserva tutte le DF ed è senza perdita è:

R1(A, B, C)	PK A
R2(C, D)	PK C
R3(F, G, H)	PK F
R4(A, F, E)	PK (A, F)

Ogni relazione è in BCNF: in ciascuna DF non banale il determinante è chiave. La decomposizione elimina le dipendenze parziali da A e F e la dipendenza transitiva tramite C.

1 luglio 2025 - Library

Esercizio 1 - Soglia

A-D

A. FK necessariamente non nulle:

- genere.bid (parte della PK);
- posizione.bid (PK);
- prestito.cid e prestito.bid (PK composta).

B. Consistency.

C. Gli indici secondari sono sempre densi.

D. posizione ha PK semplice bid, quindi è in 2NF. La DF scaffale $\rightarrow$ ripiano viola 3NF e BCNF: scaffale non è superchiave e ripiano non è primo.

E. Divisione

Il divisore contiene i libri {101, 102, 103}. Nessun cliente li ha presi tutti:

```
cid
---
(relazione vuota)
```

F. Vero/falso

1. Più copie dello stesso (titolo, autore): **Vero**, manca un vincolo `UNIQUE(titolo, autore)`.
2. Un libro con più autori: **Falso**, autore è un singolo attributo della sola tupla identificata da bid.
3. Stesso cliente e stesso libro più volte: **Falso**, la PK (cid, bid) non include la data.

Esercizio 2 - Query

1. Clienti che non hanno mai preso libri romantici

```
SELECT c.*
FROM cliente AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM prestito AS p
    JOIN genere AS g ON g.bid = p.bid
    WHERE p.cid = c.cid
    AND g.genere = 'romantico'
);
```

2. Prestito iniziato nella data di una restituzione

```
SELECT DISTINCT c.cid, c.nome
FROM cliente AS c
```

```

JOIN prestito AS iniziato ON iniziato.cid = c.cid
JOIN prestito AS restituito
  ON restituito.cid = c.cid
  AND restituito.fine_prestito = iniziato.inizio_prestito;

```

Se il docente intende due libri differenti, aggiungere AND restituito.bid <> iniziato.bid.

3. Più prestiti della media nel 2025

Versione con media calcolata su **tutti** i clienti, inclusi quelli a zero:

```

WITH conteggi AS (
  SELECT c.cid, c.nome, COUNT(p.bid) AS n
  FROM cliente AS c
  LEFT JOIN prestito AS p
    ON p.cid = c.cid
    AND p.inizio_prestito >= DATE '2025-01-01'
    AND p.inizio_prestito < DATE '2026-01-01'
  GROUP BY c.cid, c.nome
)
SELECT cid, nome, n
FROM conteggi
WHERE n > (SELECT AVG(n) FROM conteggi)
ORDER BY n DESC;

```

Se la traccia/lezione definisce la media sui soli clienti attivi, costruire conteggi da prestito con INNER JOIN.

4. Libri di Pulixi senza posizione

```

SELECT l.*
FROM libro AS l
WHERE l.autore = 'P. Pulixi'
  AND NOT EXISTS (
    SELECT 1
    FROM posizione AS pos
    WHERE pos.bid = l.bid
  );

```

5. Algebra: almeno un Carlotto e almeno un Lansdale

```

PC =  $\pi_{cid}(\text{prestito} \bowtie_{\{\text{prestito.bid}=\text{libro.bid}\}} \sigma_{\{\text{autore}='M. Carlotto'\}}(\text{libro}))$ 

```

```

PL =  $\pi_{cid}(\text{prestito} \bowtie_{\{\text{prestito.bid}=\text{libro.bid}\}} \sigma_{\{\text{autore}='J.R. Lansdale'\}}(\text{libro}))$ 

```

```

Ids = PC  $\cap$  PL
Risultato =  $\pi_{\{cid,nome\}}(\text{cliente} \bowtie \text{Ids})$ 

```

Esercizio 3 - Gerarchia (P,E)

Padre E1(a11 PK, a12), figli E2, E3, E4(a13), gerarchia parziale ed esclusiva.

Accorpamento nel padre

E1(a11 PK, a12, tipo*, a13*)

tipo $\in \{E2, E3, E4\}$ ed è nullo per istanze del solo padre. Vincolo extra-schema: a13 deve essere valorizzato esattamente per le istanze E4, se l'attributo è obbligatorio nel sottotipo.

Mantenimento di padre e figli

E1(a11 PK, a12)

E2(a11 PK/FK → E1.a11)
 E3(a11 PK/FK → E1.a11)
 E4(a11 PK/FK → E1.a11, a13)

Serve un vincolo extra-schema per l'esclusività: lo stesso a11 non deve comparire in più tabelle figlie.

Sole entità figlie

Non è applicabile: la gerarchia è parziale e si perderebbero le istanze di E1 che non appartengono ad alcun figlio.

Esercizio 4 - Grafo e revoca

Grafo iniziale:

Walter --10,60--> Walter
 Walter --20,60--> Jesse
 Jesse --30,60--> Gus
 Gus --40,60--> Mike
 Jesse --50,no60--> Mike
 Mike --60,no60--> Hector

Comando eseguito da Jesse:

REVOKE SELECT ON product FROM Gus CASCADE;

Cadono Jesse→Gus, Gus→Mike e Mike→Hector. Mike conserva il privilegio di lettura tramite Jesse→Mike al tempo 50, ma senza GO; non può quindi sostenere la concessione a Hector. Restano Walter→Jesse e Jesse→Mike oltre alla radice di Walter.

22 luglio 2025 - Festival

Esercizio 1 - Soglia

A-D

A. L'unica FK necessariamente non nulla è award.fid, perché appartiene alla PK (fid,category). screening.fid e award.aid non hanno NOT NULL e non appartengono alla PK.

B. Esclusività:

$\forall x \in A: \neg(x \in B \wedge x \in C)$

equivalentemente, $x \in B \Rightarrow x \notin C$ e viceversa. Non si può affermare che ogni A sia in B o C senza sapere che la gerarchia è totale.

C. $20.000 / 10 = 2.000$ blocchi, quindi 2.000 voci nell'indice primario.

D. Un attributo primo appartiene ad almeno una chiave candidata.

E. Risultato del LEFT JOIN

Per fid=101, title <> 'Il Padrino' è falso: nessun premio si abbina, ma il film è preservato. Il 102 non ha premi ed è preservato. Il 103 si abbina a due premi e quindi compare due volte. SELECT r.* in SQL mantiene i duplicati:

fid	title
101	Il Padrino
102	Trainspotting
103	Il padrino parte II
103	Il padrino parte II

F. Effetto del DELETE

award contiene una riga con fid=101, ma la sua FK verso film non ha ON DELETE CASCADE. Il DELETE è quindi rifiutato per integrità referenziale. L'istruzione è atomica: non viene cancellato il film e non vengono cancellate neppure eventuali proiezioni, nonostante screening.fid abbia CASCADE.

Esercizio 2 - Query

1. Film con il maggior numero di premi

```
WITH conteggi AS (  
    SELECT f.fid, f.title, COUNT(a.category) AS n  
    FROM film AS f  
    LEFT JOIN award AS a ON a.fid = f.fid  
    GROUP BY f.fid, f.title  
)  
SELECT *  
FROM conteggi  
WHERE n = (SELECT MAX(n) FROM conteggi);
```

2. Coppie di artisti, stesso premio, anni consecutivi, film diversi

```
SELECT DISTINCT  
    ar1.aid, ar1.name,  
    ar2.aid, ar2.name,  
    a1.category, a1.year, a2.year  
FROM award AS a1  
JOIN award AS a2  
    ON a2.category = a1.category  
    AND a2.year = a1.year + 1  
    AND a2.fid <> a1.fid  
    AND a2.aid <> a1.aid  
JOIN artist AS ar1 ON ar1.aid = a1.aid  
JOIN artist AS ar2 ON ar2.aid = a2.aid;
```

Se sono ammesse due vittorie consecutive dello stesso artista, eliminare a2.aid <> a1.aid.

3. Artisti con premi in ogni categoria presente

```
SELECT ar.*  
FROM artist AS ar  
WHERE NOT EXISTS (  
    SELECT 1  
    FROM (SELECT DISTINCT category FROM award) AS c  
    WHERE NOT EXISTS (  
        SELECT 1  
        FROM award AS a  
        WHERE a.aid = ar.aid  
        AND a.category = c.category  
    )  
);
```

4. Film proiettati in più di tre venue diverse

```
SELECT f.fid, f.title  
FROM film AS f  
JOIN screening AS s ON s.fid = f.fid  
GROUP BY f.fid, f.title  
HAVING COUNT(DISTINCT s.venue) > 3;
```

5. Algebra ottimizzata: film senza premiere

Premiere = $\pi_{fid}(\sigma_{\{is_premiere=true\}}(screening))$

Senza = $\pi_{fid}(film) - Premiere$
Risultato = $\pi_{\{fid,title\}}(film \bowtie Senza)$

La selezione e la proiezione sono spinte prima della differenza/join.

Esercizio 3 - Reverse engineering ER

- E1: entità, identificatore A11, attributo A12.
- E2: entità, identificatore composto (A21, A22). A23 è FK opzionale verso E1: associazione N:1, E2 partecipa 0..1, E1 0..N.
- E3: entità, identificatore A31. A32 è FK opzionale verso E1: associazione N:1 con le stesse cardinalità.
- E4: entità, identificatore A41, attributo A42. La FK composta opzionale (A43, A44) la collega a E2: E4 0..1, E2 0..N.
- E5: la PK (A51, A52, A53) è esattamente l'unione delle FK verso E1 ed E2 e non ci sono attributi descrittivi. Si minimizza E5 in un'associazione N:M fra E1 ed E2; i due lati hanno cardinalità 0..N in assenza di altri vincoli.

Nota SQL: con una FK composta nullable e MATCH SIMPLE, un solo componente nullo basta a evitare il controllo. A livello concettuale conviene imporre che A43, A44 siano entrambi nulli oppure entrambi valorizzati.

Esercizio 4 - Cavallo di Troia

Una procedura apparentemente utile contiene codice nascosto. Un utente che ha diritto di leggere il dato segreto e di scrivere su un oggetto accessibile all'attaccante esegue la procedura; questa copia il segreto nell'oggetto meno protetto. L'attaccante lo legge senza aver mai ricevuto accesso diretto alla fonte. È un limite delle politiche discrezionali, che non controllano il flusso dell'informazione dopo l'accesso; le politiche mandatorie mirano a impedirlo.

Errori che farebbero perdere punti in più appelli

1. Mettere in WHERE il filtro della tabella destra di un LEFT JOIN e perdere gli oggetti con zero.
2. Usare COUNT(*) dopo un join esterno.
3. Restituire un solo massimo con LIMIT 1 quando sono possibili ex aequo.
4. Dimenticare DISTINCT in conteggi di categorie, venue o certificati.
5. Confondere una FK nullable con una FK obbligatoria perché "semanticamente dovrebbe esserlo". Conta il DDL.
6. Dire che un CASCADE avviene anche quando un'altra FK fa fallire l'intera istruzione.
7. Nell'algebra, usare tipo=A AND tipo=B invece dell'intersezione di due insiemi di identificatori.
8. Nella revoca, ignorare fonti indipendenti, grant option o timestamp.
9. Definire 3NF come "assenza di dipendenze transitive" senza verificare superchiavi e attributi primi.
10. Tradurre una gerarchia parziale usando solo le tabelle figlie: si perdono le istanze del solo padre.